# Analysis of GenAI Programs

The two GenAI programs used were Gemini and Claude. Both GenAIs were accessed using their desktop applications.

## Prompt used for both GenAIs:

Write a complete, object-oriented C++ program that prioritizes emails for a busy company CEO. The program must use objects, not functions, to implement the following requirements.
Core Data Structure:
Implement a priority queue using a list-based MaxHeap.
You must create all objects and the MaxHeap entirely from scratch. Do not include or use any pre-existing heap modules or standard library heap algorithms.
Email Priority Rules:
Place incoming emails into the queue based on the following sender category hierarchy:
Boss (read first)
Subordinate (read next)
Peer (read next)
ImportantPerson (read next)
OtherPerson (read last)
Tie-breaker rule: If there is more than one email from a sender category, the newest email must be read first (not the oldest).
Command Processing:
The program will read test files containing the following commands (the order and number of commands will vary):
EMAIL <sender category>, <subject line>, <date>: Adds the email to the priority queue.
EMAIL is followed by a space, and the remaining fields are comma-delimited. Assume the subject line may contain spaces but not commas, and the date is exactly in MM-DD-YYYY format.
NEXT: Displays the highest priority email without removing it. The output format must be:
Sender:
Subject:
Date:
READ: Removes the highest priority email from the queue (the CEO has dealt with it).
COUNT: Displays the current count of unread emails.
Edge cases to handle: If two NEXT commands occur in a row without a READ, display the same email both times. If two READ commands occur in a row, delete the top email from the queue without displaying it. Gracefully handle NEXT or READ commands when the queue is completely empty.

**Gemini generated code:**

```
/*
 * PROLOGUE COMMENTS
 * Program Name: EECS 348 Assignment 3 - CEO Email Prioritizer[cite: 1]
 * Description: A C++ object-oriented program that prioritizes emails
for a busy company CEO.[cite: 1]
 *              It reads commands from a test file to queue and process
emails.[cite: 2]
 *              The program implements a MaxHeap priority queue using a
list-based implementation from scratch.[cite: 1]
 * Inputs: A text file containing EMAIL, NEXT, READ, and COUNT
commands.[cite: 2]
 * Outputs: Terminal output displaying the next highest priority email
and the count of unread emails.[cite: 2]
 * Collaborators: None[cite: 4]
 * Other Sources: Code structure and logic conceptualization assisted by
Gemini (AI).[cite: 4]
 * Author's Full Name: Brayden Nelson[cite: 6]
 * Creation Date: October 1, 2026
 * Revision Date: October 1, 2026
 */

#include <iostream> // Includes standard I/O library to handle terminal
input and output
#include <fstream>  // Includes file stream library to open and read the
test file
#include <vector>   // Includes vector library to act as the dynamic
list for our list-based MaxHeap[cite: 1]
#include <string>   // Includes string library to handle command
strings, subjects, and sender categories
#include <sstream>  // Includes string stream library to help parse the
comma-delimited input lines

/*
 * MAJOR BLOCK: String Utilities Class
 * This class acts as a utility object to encapsulate string
manipulation methods.
 * It avoids using standalone functions to adhere to strict
```

```
object-oriented requirements.[cite: 1]
 */
class StringUtils { // Declare the StringUtils class
public: // Make the methods public so other objects can use them
    std::string trim(std::string str) { // Method to remove leading and
trailing whitespace from a string
        size_t first = str.find_first_not_of(" \t\r\n"); // Find the
index of the first non-whitespace character
        if (first == std::string::npos) return ""; // If no
non-whitespace characters exist, return an empty string
        size_t last = str.find_last_not_of(" \t\r\n"); // Find the index
of the last non-whitespace character
        return str.substr(first, (last - first + 1)); // Extract and
return the substring without the outer whitespace
    } // End of trim method
}; // End of StringUtils class

/*
 * MAJOR BLOCK: Email Class
 * This class represents a single email object with properties for
sender, subject, and date.
 * It includes logic to calculate a numerical priority for the sender
category and parse the date.
 * This allows the MaxHeap to easily compare which email should be read
first.
 */
class Email { // Declare the Email class to hold email data
public: // Make members public for easy access by the priority queue
    std::string sender; // Declare a string variable to store the sender
category (e.g., Boss, Peer)[cite: 2]
    std::string subject; // Declare a string variable to store the
subject line[cite: 2]
    std::string dateStr; // Declare a string variable to store the exact
MM-DD-YYYY date[cite: 2]
    int priorityScore; // Declare an integer to store the calculated
hierarchy score of the sender
    int dateScore; // Declare an integer to store the YYYYMMDD numeric
representation of the date for tie-breakers
```

```
    Email(std::string s, std::string sub, std::string d) { //
Parameterized constructor to initialize the email
        StringUtils util; // Instantiate the utility object to trim
inputs
        sender = util.trim(s); // Trim and assign the sender category
        subject = util.trim(sub); // Trim and assign the subject line
        dateStr = util.trim(d); // Trim and assign the date string
        priorityScore = calculateSenderPriority(sender); // Call method
to map the sender to a numeric score
        dateScore = calculateDateScore(dateStr); // Call method to
convert the date string to an integer for sorting
    } // End of constructor

    Email() { // Default constructor for empty initialization
        sender = ""; // Set sender to empty string
        subject = ""; // Set subject to empty string
        dateStr = ""; // Set date to empty string
        priorityScore = -1; // Set default invalid priority score
        dateScore = -1; // Set default invalid date score
    } // End of default constructor

private: // Private access modifier for helper calculation methods
    int calculateSenderPriority(std::string cat) { // Method to map
string category to integer priority
        // Boss is the highest priority, read first[cite: 2]
        if (cat == "Boss") return 5; // Return score 5 for Boss
        // Subordinate is read next[cite: 2]
        if (cat == "Subordinate") return 4; // Return score 4 for
Subordinate
        // Peer is read next[cite: 2]
        if (cat == "Peer") return 3; // Return score 3 for Peer
        // ImportantPerson is read next[cite: 2]
        if (cat == "ImportantPerson" || cat == "Important Person")
return 2; // Return score 2 for Important Person
        // OtherPerson is read last[cite: 2]
        if (cat == "OtherPerson" || cat == "Other Person") return 1; //
Return score 1 for Other Person
        return 0; // Return 0 for unrecognized categories as a fail-safe
    } // End of calculateSenderPriority method
```

```
    int calculateDateScore(std::string d) { // Method to parse
MM-DD-YYYY into YYYYMMDD integer
        if (d.length() < 10) return 0; // Guard against malformed date
strings that are too short
        std::string month = d.substr(0, 2); // Extract the two-digit
month
        std::string day = d.substr(3, 2); // Extract the two-digit day
        std::string year = d.substr(6, 4); // Extract the four-digit
year
        std::string combined = year + month + day; // Concatenate in
year-month-day order for numerical sorting
        return std::stoi(combined); // Convert the concatenated string
to an integer and return it
    } // End of calculateDateScore method
}; // End of Email class

/*
 * MAJOR BLOCK: MaxHeap Class
 * Implements a priority queue using a list-based (vector) MaxHeap
entirely from scratch.[cite: 1]
 * It does not include any pre-existing heap modules.[cite: 1]
 * The sorting prioritizes higher sender categories, using the newest
date as a tie-breaker.[cite: 2]
 */
class MaxHeap { // Declare the MaxHeap class
private: // Private access modifier for internal heap logic and storage
    std::vector<Email> heapArray; // Use a dynamic array (list-based) to
store heap elements[cite: 1]

    bool isLessThan(Email a, Email b) { // Helper method to compare two
emails and determine strict lower priority
        if (a.priorityScore != b.priorityScore) { // Check if sender
priorities are different
            return a.priorityScore < b.priorityScore; // Return true if
'a' has a lower category priority than 'b'
        } // End if
        // Tie-breaker rule: if there is more than one email from a
sender category, the newest email must be read first[cite: 2]
```

```
        return a.dateScore < b.dateScore; // Return true if 'a' has an
older date (lower numerical value) than 'b'
    } // End of isLessThan method

    void heapifyUp(int index) { // Method to restore max-heap property
by bubbling elements upward
        while (index > 0) { // Loop as long as the current index is not
the root node (0)
            int parent = (index - 1) / 2; // Calculate the parent node's
integer index
            if (isLessThan(heapArray[parent], heapArray[index])) { //
Check if the parent is lower priority than the current node
                Email temp = heapArray[parent]; // Temporarily store the
parent element
                heapArray[parent] = heapArray[index]; // Move the
current higher-priority element up to the parent slot
                heapArray[index] = temp; // Move the parent element down
to the current slot
                index = parent; // Update the current index to continue
checking upwards through the tree
            } else { // If the parent is greater than or equal to the
current node
                break; // The heap property is satisfied, exit the loop
            } // End if-else
        } // End while loop
    } // End of heapifyUp method

    void heapifyDown(int index) { // Method to restore max-heap property
by sinking elements downward
        int size = heapArray.size(); // Get the current number of
elements in the heap array
        while (true) { // Infinite loop broken manually when heap
property is fully restored
            int leftChild = 2 * index + 1; // Calculate the index of the
left child node
            int rightChild = 2 * index + 2; // Calculate the index of
the right child node
            int largest = index; // Assume the current node is the
largest priority to start
```

```
            if (leftChild < size && isLessThan(heapArray[largest],
heapArray[leftChild])) { // If left child exists and has higher priority
                largest = leftChild; // Update largest index pointer to
be the left child
            } // End if

            if (rightChild < size && isLessThan(heapArray[largest],
heapArray[rightChild])) { // If right child exists and has higher
priority
                largest = rightChild; // Update largest index pointer to
be the right child
            } // End if

            if (largest != index) { // If the largest is no longer the
current index (meaning a child is larger)
                Email temp = heapArray[index]; // Temporarily store the
current element
                heapArray[index] = heapArray[largest]; // Swap the
larger child into the current slot
                heapArray[largest] = temp; // Place the current element
into the child's lower-priority slot
                index = largest; // Update the index to the child's
position to continue checking downward
            } else { // If the current element is larger than both
children
                break; // The heap property is satisfied, exit the loop
            } // End if-else
        } // End while loop
    } // End of heapifyDown method

public: // Public access modifier for interface methods exposing queue
functionality
    void insert(Email email) { // Method to add a new email object to
the priority queue
        heapArray.push_back(email); // Append the new email to the end
of the dynamic array
        heapifyUp(heapArray.size() - 1); // Bubble the newly added
element up to its correct hierarchical position
```

```
    } // End of insert method

    Email peekNext() { // Method to view the highest priority email
without removing it from the queue
        if (heapArray.empty()) { // Check if the queue currently has no
emails
            return Email("", "", ""); // Return an empty email object to
gracefully handle a NEXT command on an empty queue[cite: 2]
        } // End if
        return heapArray[0]; // Return the root element (index 0), which
always holds the highest priority item
    } // End of peekNext method

    void extractTop() { // Method to delete the highest priority email
once read
        if (heapArray.empty()) { // Check if the queue is already empty
            return; // Return and do nothing to gracefully handle a READ
command on an empty queue[cite: 2]
        } // End if
        heapArray[0] = heapArray.back(); // Overwrite the root element
with the last element in the list array
        heapArray.pop_back(); // Remove the last element from the array,
effectively deleting the previous top email
        if (!heapArray.empty()) { // If the heap is not completely empty
after the removal
            heapifyDown(0); // Sink the new root element down to its
correct position to restore the heap
        } // End if
    } // End of extractTop method

    int getCount() { // Method to get the total number of unread emails
        return heapArray.size(); // Return the integer size of the
underlying dynamic array
    } // End of getCount method
}; // End of MaxHeap class

/*
 * MAJOR BLOCK: CEOInboxSystem Class
 * This class encapsulates the entire file parsing, command execution,
```

```
and formatting logic.
 * It reads the given test file, splits strings based on delimiters, and
interacts
 * with the MaxHeap to manage the CEO's emails according to assignment
constraints.
 */
class CEOInboxSystem { // Declare the CEOInboxSystem class
private: // Private access modifier for internal system attributes
    MaxHeap inboxQueue; // Instantiate the custom MaxHeap object to
store and sort the emails

public: // Public access modifier for primary execution methods
    void processFile(std::string filename) { // Method to open and read
commands from the specified text file
        std::ifstream fileStream(filename); // Open the file stream
using the provided filename
        if (!fileStream.is_open()) { // Check if the file failed to open
successfully
            std::cout << "Could not open test file: " << filename <<
std::endl; // Output an error message to the terminal
            return; // Exit the method early if the file is unreadable
        } // End if

        std::string line; // Declare a string to store each line read
from the file sequentially
        StringUtils util; // Instantiate the string utility object for
trimming whitespace

        while (std::getline(fileStream, line)) { // Loop continuously,
reading the file line by line until the end
            line = util.trim(line); // Remove any leading or trailing
whitespace from the current line
            if (line.empty()) continue; // Skip execution for blank
lines to prevent unexpected parsing errors

            if (line.substr(0, 5) == "EMAIL") { // Check if the string
begins with the "EMAIL" command[cite: 2]
                processEmailCommand(line); // Call helper method to
parse the comma-delimited data and add the email
```

```
            } // End if
            else if (line == "NEXT") { // Check if the string matches
the "NEXT" command exactly[cite: 2]
                processNextCommand(); // Call helper method to display
the highest priority unread email
            } // End else-if
            else if (line == "READ") { // Check if the string matches
the "READ" command exactly[cite: 2]
                processReadCommand(); // Call helper method to delete
the top email from the priority queue
            } // End else-if
            else if (line == "COUNT") { // Check if the string matches
the "COUNT" command exactly[cite: 2]
                processCountCommand(); // Call helper method to output
the total number of emails remaining
            } // End else-if
        } // End while loop

        fileStream.close(); // Close the file stream to release the
system memory resources
    } // End of processFile method

private: // Private access modifier for internal command processing
helper routines
    void processEmailCommand(std::string line) { // Method to parse an
EMAIL command line and enqueue the result
        StringUtils util; // Instantiate string utility object
        std::string content = line.substr(6); // Remove the "EMAIL "
prefix (first 6 characters)[cite: 2]

        size_t firstComma = content.find(','); // Find the index of the
first comma separating the sender and subject[cite: 2]
        size_t secondComma = content.find(',', firstComma + 1); // Find
the index of the second comma separating subject and date[cite: 2]

        if (firstComma == std::string::npos || secondComma ==
std::string::npos) { // Ensure both comma delimiters exist in the line
            return; // Exit the method gracefully if the file formatting
is invalid
```

```
        } // End if

        std::string senderStr = util.trim(content.substr(0,
firstComma)); // Extract and trim the sender category substring
        std::string subjectStr = util.trim(content.substr(firstComma +
1, secondComma - firstComma - 1)); // Extract and trim the subject line
substring
        std::string dateStr = util.trim(content.substr(secondComma +
1)); // Extract and trim the exact MM-DD-YYYY date substring[cite: 2]

        Email newEmail(senderStr, subjectStr, dateStr); // Create a new
Email object using the three parsed string values
        inboxQueue.insert(newEmail); // Insert the fully constructed
Email object into the custom MaxHeap
    } // End of processEmailCommand method

    void processNextCommand() { // Method to strictly format and handle
the NEXT terminal command
        Email top = inboxQueue.peekNext(); // Retrieve the highest
priority email without removing it (handles consecutive NEXTs)[cite: 2]
        if (top.sender == "") { // Check if the returned email has an
empty sender (indicating an empty queue)
            std::cout << "The inbox is empty. No email to display." <<
std::endl; // Print a graceful error message instead of crashing
        } else { // If an email successfully exists in the queue
            std::cout << "Next email:" << std::endl; // Print the header
line as required by the sample test output[cite: 3]
            std::cout << "Sender: " << top.sender << std::endl; // Print
the dynamically prioritized sender category[cite: 3]
            std::cout << "Subject: " << top.subject << std::endl; //
Print the corresponding subject line[cite: 3]
            std::cout << "Date: " << top.dateStr << std::endl; // Print
the corresponding formatted date string[cite: 3]
        } // End if-else
    } // End of processNextCommand method

    void processReadCommand() { // Method to handle the READ command
when the CEO processes an email
        inboxQueue.extractTop(); // Extract and delete the top element
```

```
(deals with consecutive READ commands gracefully without crashing)[cite:
2]
    } // End of processReadCommand method

    void processCountCommand() { // Method to handle the COUNT command
and display remaining emails
        int count = inboxQueue.getCount(); // Call the heap method to
retrieve the number of elements
        std::cout << "There are " << count << " emails to read." <<
std::endl; // Display the current count matching the assignment output
format[cite: 3]
    } // End of processCountCommand method
}; // End of CEOInboxSystem class

/*
 * MAJOR BLOCK: Main Application Entry Point
 * This block contains the main method required by the C++ compiler to
launch the executable.
 * It parses command line arguments to get the test file name and
delegates the application
 * logic to the CEOInboxSystem object.
 */
int main(int argc, char* argv[]) { // Define the main execution function
taking in argument count and character arrays
    std::string filename; // Declare a string to hold the target text
file path
    if (argc >= 2) { // Check if the user passed a file argument via the
execution command line
        filename = argv[1]; // Assign the first argument as the test
file name string
    } else { // If no command line arguments were provided by the user
        std::cout << "Enter the name of the test file: "; // Prompt the
user to type the filename interactively
        std::cin >> filename; // Read the user's terminal input into the
filename variable
    } // End if-else

    CEOInboxSystem systemRunner; // Instantiate the primary object
representing the overarching CEO email system
```

```
    systemRunner.processFile(filename); // Call the object's processing
method using the determined filename

    return 0; // Return integer zero indicating successful application
execution
} // End of main function
```

**Claude generated code:**

```
/*
 * PROLOGUE
 * Program Name : EECS 348 Assignment 3 - CEO Email Prioritizer
 * Description  : Object-oriented C++ program that prioritizes emails
for a busy
 *                CEO. Emails are stored in a hand-written, list-based
MaxHeap
 *                (priority queue). Priority is decided by sender
category
 *                (Boss > Subordinate > Peer > ImportantPerson >
OtherPerson);
 *                within the same category the NEWEST email is read
first.
 * Inputs       : The program prompts for the name of a text file, which
contains lines of:
 *                  EMAIL <sender category>,<subject line>,<date
MM-DD-YYYY>
 *                  NEXT   (show top-priority email, do not remove it)
 *                  READ   (remove top-priority email)
 *                  COUNT  (show number of unread emails)
 * Collaborators / Sources : Code generated by claude from the
assignment prompt. No human collaborators.
 * Author       : Brayden Nelson
 * Creation Date: October 1, 2026
 */

#include <iostream>   // Provides std::cout and std::cin for terminal
I/O
#include <fstream>    // Provides std::ifstream for reading the test
```

```
file
#include <sstream>    // Provides std::stringstream for splitting
comma-delimited text
#include <string>     // Provides std::string for text handling
#include <cctype>     // Provides std::toupper for case-insensitive
command matching
#include <vector>     // Provides std::vector, used ONLY as the plain
list storing heap items

//
--------------------------------------------------------------------------
---
// CLASS: Email
// Purpose: Holds one email and knows how to compare its own priority
with
//          another email.
//
--------------------------------------------------------------------------
---
class Email {                                       // Begin definition of
the Email class
private:                                            // Data members are
hidden from outside code
    std::string senderCategory;                     // Sender category
text, e.g. "Boss"
    std::string subject;                            // Subject line text
(may contain spaces)
    std::string date;                               // Date text in
MM-DD-YYYY format
    int categoryRank;                               // Numeric rank of
category (higher = more urgent)
    int dateValue;                                  // Date converted to
YYYYMMDD so newer = larger
    long arrivalOrder;                              // Order the email
arrived (used only to break exact ties)

    // Converts a category string into a numeric rank (5 = Boss ... 1 =
OtherPerson)
    static int rankFromCategory(const std::string& cat) {  // Helper
```

```
belonging to the class
        if (cat == "Boss") return 5;              // Boss is read first,
so highest rank
        if (cat == "Subordinate") return 4;       // Subordinate is read
next
        if (cat == "Peer") return 3;              // Peer is read next
        if (cat == "ImportantPerson") return 2;   // ImportantPerson is
read next
        return 1;                                 // OtherPerson (or
anything else) is read last
    }                                             // End of
rankFromCategory

    // Converts "MM-DD-YYYY" into the integer YYYYMMDD so dates compare
numerically
    static int valueFromDate(const std::string& d) {       // Helper
belonging to the class
        if (d.size() < 10) return 0;              // Malformed date:
treat as oldest possible
        int month = std::stoi(d.substr(0, 2));    // Characters 0-1 hold
the month
        int day = std::stoi(d.substr(3, 2));      // Characters 3-4 hold
the day
        int year = std::stoi(d.substr(6, 4));     // Characters 6-9 hold
the year
        return year * 10000 + month * 100 + day;  // Combine so year
dominates, then month, then day
    }                                             // End of
valueFromDate

public:                                           // Members below are
usable by other classes
    Email() : categoryRank(0), dateValue(0), arrivalOrder(0) {}  //
Default constructor with zeroed numbers

    // Constructor that builds a fully populated email
    Email(const std::string& cat, const std::string& subj,
          const std::string& dt, long order)      // Parameters:
category, subject, date, arrival number
```

```
        : senderCategory(cat), subject(subj), date(dt),   // Store the
three text fields
          categoryRank(rankFromCategory(cat)),    // Compute and store
the category rank
          dateValue(valueFromDate(dt)),           // Compute and store
the numeric date
          arrivalOrder(order) {}                  // Store arrival
order; constructor body is empty

    // Returns true if THIS email should be read before the OTHER email
    bool hasHigherPriorityThan(const Email& other) const {  //
Comparison used by the heap
        if (categoryRank != other.categoryRank)   // If categories
differ...
            return categoryRank > other.categoryRank;  // ...the
higher-ranked category wins
        if (dateValue != other.dateValue)         // Same category but
different dates...
            return dateValue > other.dateValue;   // ...the NEWER date
wins
        return arrivalOrder < other.arrivalOrder; // Same category and
date: earlier arrival wins (stable)
    }                                             // End of
hasHigherPriorityThan

    // Prints the email in the required output format
    void display() const {                        // Show this email on
the terminal
        std::cout << "Sender: " << senderCategory << "\n";  // Print the
sender line
        std::cout << "Subject: " << subject << "\n";        // Print the
subject line
        std::cout << "Date: " << date << "\n";              // Print the
date line
    }                                             // End of display
};                                                // End of Email class

//
--------------------------------------------------------------------------
```

```
---
// CLASS: MaxHeap
// Purpose: Hand-written list-based binary max-heap of Email objects,
used as a
//          priority queue. Written from scratch (no
std::priority_queue,
//          std::push_heap, std::pop_heap, or std::make_heap).
//
-------------------------------------------------------------------------
---
class MaxHeap {                                     // Begin definition of
the MaxHeap class
private:                                            // Internal storage
and helpers are hidden
    std::vector<Email> items;                       // The list: node i
has children 2i+1 and 2i+2

    // Moves the item at index i upward until the heap property is
restored
    void siftUp(size_t i) {                         // Used after an
insert
        while (i > 0) {                             // Stop when we reach
the root
            size_t parent = (i - 1) / 2;            // Compute the
parent's index
            if (items[i].hasHigherPriorityThan(items[parent])) {  //
Child beats parent?
                Email temp = items[i];              // Save the child in a
temporary variable
                items[i] = items[parent];           // Copy the parent
down into the child's slot
                items[parent] = temp;               // Put the saved child
into the parent's slot
                i = parent;                         // Continue checking
from the parent's position
            } else {                                // Child does not beat
parent...
                break;                              // ...so the heap
property holds; stop
```

```
            }                                           // End of if/else
        }                                               // End of while
    }                                                   // End of siftUp

    // Moves the item at index i downward until the heap property is
restored
    void siftDown(size_t i) {                           // Used after removing
the root
        size_t n = items.size();                        // Number of items
currently in the heap
        while (true) {                                  // Loop until the item
is in a valid spot
            size_t left = 2 * i + 1;                    // Index of the left
child
            size_t right = 2 * i + 2;                   // Index of the right
child
            size_t best = i;                            // Assume current node
has the highest priority
            if (left < n &&
items[left].hasHigherPriorityThan(items[best]))     // Left child exists
and beats best?
                best = left;                            // Left child becomes
the best candidate
            if (right < n &&
items[right].hasHigherPriorityThan(items[best])) // Right child exists
and beats best?
                best = right;                           // Right child becomes
the best candidate
            if (best == i) break;                       // Current node
already beats its children; done
            Email temp = items[i];                      // Save the current
node in a temporary variable
            items[i] = items[best];                     // Move the stronger
child up
            items[best] = temp;                         // Move the saved node
down
            i = best;                                   // Continue from the
child's position
        }                                               // End of while
```

```
    }                                               // End of siftDown

public:                                             // Operations other
classes may call
    // Adds an email to the heap in O(log n)
    void insert(const Email& e) {                   // Parameter: the
email to add
        items.push_back(e);                         // Place it at the end
of the list
        siftUp(items.size() - 1);                   // Bubble it up to its
correct position
    }                                               // End of insert

    // Returns true when the heap holds no emails
    bool isEmpty() const {                          // Query used before
peek/remove
        return items.empty();                       // True if the list
has zero elements
    }                                               // End of isEmpty

    // Returns how many emails are currently in the heap
    size_t size() const {                           // Query used by COUNT
        return items.size();                        // Number of elements
in the list
    }                                               // End of size

    // Returns a reference to the top-priority email without removing it
(O(1)); caller must check isEmpty first
    const Email& peek() const {                     // Used by NEXT
        return items[0];                            // The root of the
heap is always the highest priority
    }                                               // End of peek

    // Removes the top-priority email in O(log n); does nothing if empty
    void removeTop() {                              // Used by READ
        if (items.empty()) return;                  // Nothing to remove
when the heap is empty
        items[0] = items.back();                    // Move the last item
into the root slot
```

```
        items.pop_back();                           // Shrink the list by
one
        if (!items.empty()) siftDown(0);            // Restore the heap
property from the root downward
    }                                               // End of removeTop
};                                                  // End of MaxHeap
class

//
-------------------------------------------------------------------------
---
// CLASS: CommandProcessor
// Purpose: Reads command lines, parses them, and applies them to a
MaxHeap.
//          All program behavior lives in this object.
//
-------------------------------------------------------------------------
---
class CommandProcessor {                            // Begin definition of
the CommandProcessor class
private:                                            // Internal state is
hidden
    MaxHeap inbox;                                  // The CEO's inbox,
stored as a priority queue
    long nextArrival;                               // Counter used to
number emails as they arrive

    // Removes leading/trailing spaces, tabs, and carriage returns from
a string
    static std::string trim(const std::string& s) {  // Returns a
cleaned copy
        const std::string ws = " \t\r\n";           // Characters
considered whitespace
        size_t start = s.find_first_not_of(ws);     // Find first
non-whitespace character
        if (start == std::string::npos) return "";// String was all
whitespace: return empty
        size_t end = s.find_last_not_of(ws);        // Find last
non-whitespace character
```

```
        return s.substr(start, end - start + 1);  // Return only the
middle portion
    }                                             // End of trim

    // Returns an upper-case copy of a string so commands match
regardless of case
    static std::string toUpper(std::string s) {   // Parameter is a copy
we may modify
        for (size_t i = 0; i < s.size(); i++)     // Visit every
character
            s[i] = (char)std::toupper((unsigned char)s[i]);  // Convert
it to upper case
        return s;                                 // Hand back the
converted copy
    }                                             // End of toUpper

    // Handles an "EMAIL <category>,<subject>,<date>" line
    void handleEmail(const std::string& rest) {   // Parameter: text
after "EMAIL "
        std::stringstream ss(rest);               // Wrap the text so we
can split on commas
        std::string category, subject, date;      // Storage for the
three comma-delimited fields
        std::getline(ss, category, ',');          // Read up to the
first comma: category
        std::getline(ss, subject, ',');           // Read up to the
second comma: subject
        std::getline(ss, date, ',');              // Read the remainder:
date
        inbox.insert(Email(trim(category), trim(subject), trim(date),
nextArrival++));  // Build and enqueue the email
    }                                             // End of handleEmail

    // Handles the NEXT command: show top email without removing it
    void handleNext() {                           // No parameters
        if (inbox.isEmpty()) {                    // Is there nothing to
read?
            std::cout << "There are no emails to read.\n";  //
Gracefully report empty inbox
```

```
            return;                                 // Nothing more to do
        }                                           // End of empty check
        std::cout << "Next email:\n";               // Print the header
line
        inbox.peek().display();                     // Print sender,
subject, and date of the top email
    }                                               // End of handleNext

    // Handles the READ command: discard top email silently
    void handleRead() {                             // No parameters
        inbox.removeTop();                          // Remove top email
(safe no-op if empty)
    }                                               // End of handleRead

    // Handles the COUNT command: print number of unread emails
    void handleCount() {                            // No parameters
        std::cout << "There are " << inbox.size() << " emails to
read.\n";  // Print the count message
    }                                               // End of handleCount

public:                                             // Operations other
code may call
    CommandProcessor() : nextArrival(0) {}          // Constructor:
arrival counter starts at zero

    // Reads every line from the given stream and executes it
    void run(std::istream& in) {                    // Parameter: input
stream (file or stdin)
        std::string line;                           // Holds one line of
input at a time
        bool firstLine = true;                      // Tracks whether we
are on the first line (for BOM removal)
        while (std::getline(in, line)) {            // Loop until the
stream has no more lines
            if (firstLine && line.compare(0, 3, "\xEF\xBB\xBF") == 0)
// Does the file start with a UTF-8 BOM?
                line.erase(0, 3);                   // Yes: remove the
invisible 3-byte mark
            firstLine = false;                      // Any later line is
```

```
no longer the first
            line = trim(line);                      // Strip stray
whitespace / Windows line endings
            if (line.empty()) continue;             // Skip blank lines
            size_t split = line.find_first_of(" \t");  // Find the end
of the command word
            std::string command = toUpper(line.substr(0, split));  //
Command word, upper-cased
            std::string rest = (split == std::string::npos) ? "" :
trim(line.substr(split));  // Text after the command
            if (command == "EMAIL") {               // Is it the EMAIL
command?
                handleEmail(rest);                  // Yes: pass the
fields along
            } else if (command == "NEXT") {         // Is it the NEXT
command?
                handleNext();                       // Yes: show top email
            } else if (command == "READ") {         // Is it the READ
command?
                handleRead();                       // Yes: remove top
email
            } else if (command == "COUNT") {        // Is it the COUNT
command?
                handleCount();                      // Yes: show unread
count
            } else {                                // Anything else is
not a valid command
                std::cerr << "Warning: unrecognized line ignored: " <<
line << "\n";  // Report it on stderr
            }                                       // End of command
dispatch
        }                                           // End of while
        std::cout.flush();                          // Make sure all
output is written before returning
    }                                               // End of run
};                                                  // End of
CommandProcessor class

// Program entry point (required by C++; it only creates and runs the
```

```
object)
int main() {                                        // No command-line
arguments are needed
    std::string fileName;                           // Will hold the input
file name typed by the user
    std::cout << "Enter the name of the input file: ";  // Prompt the
user for the file name
    std::getline(std::cin, fileName);               // Read the whole
typed line (allows spaces in names)
    CommandProcessor processor;                     // Create the object
that does all the work
    std::ifstream file(fileName);                   // Try to open the
named input file
    if (!file) {                                    // Did opening fail?
        std::cerr << "Error: cannot open file " << fileName << "\n";  //
Tell the user which file failed
        return 1;                                   // Exit with an error
code
    }                                               // End of failure
check
    processor.run(file);                            // Process every
command in the file
    return 0;                                       // Report successful
completion
}                                                   // End of main
```

## Analysis:

**Correctness -**

- **Claude:** The program effectively handles all standard inputs and correctly translates the date into a numeric YYYYMMDD format to prioritize the newest emails. It accounts for edge cases like exact ties by utilizing an arrivalOrder variable, ensuring stable sorting. It also safely handles common input file formatting anomalies like trailing spaces and invisible UTF-8 BOM characters.
- **Gemini:** The code successfully converts sender categories and dates to numeric values to build the heap. However, it lacks a strict tie-breaking mechanism for exact ties beyond the date, and handles empty queue NEXT commands by returning an empty Email object, which is less robust than Claude's approach of checking if the queue is empty first.

**Execution Time -**

- **Claude:** The heap insertion and deletion operations scale at an optimal Olog(n)time complexity. Claude passes objects by constant reference (e.g., const Email&), avoiding the costly overhead of duplicating string values in memory.
- **Gemini:** While the structural time complexity is also Olog(n) for heap operations, Gemini passes Email objects by value in methods like insert (Email email) and isLessThan(Email a, Email b). This creates hidden constant-time overheads due to string copying. Gemini also repeatedly instantiates its StringUtils class on every string operation.

**Space Complexity -**

- **Claude:** The program uses a standard std::vector for its list-based MaxHeap, resulting in an expected O(n) space complexity. By passing objects by reference, it keeps the call stack lean and uses constant auxiliary space O(1).
- **Gemini:** The structural space complexity is O(n). However, due to pass-by-value methods, each function call duplicates Email variables, unnecessarily inflating the memory consumed by the call stack.

**Maintainability -**

- **Claude:** The code includes an excellent prologue and meticulous line-by-line comments. It appropriately encapsulates helper methods as static functions within the classes they belong to, producing highly readable object-oriented architecture.
- **Gemini:** While it contains the required prologue and line-by-line comments, the architecture forces string utilities into a standalone `StringUtils` object simply to mimic object-oriented rules. This disjointed design reduces overall readability and increases the likelihood of future bugs.

## Code Selection:

I selected Claude as the basis for the assignment. It is fundamentally more efficient because it passes parameters by reference instead of by value, reducing unnecessary memory allocation overhead. Claude also provides better edge-case handling for exact ties via the arrivalOrder logic and strips hidden UTF-8 BOM characters that frequently cause parsing errors when reading files.

## Code Improvement:

- **Correctness**: I added validation checks inside the valueFromDate method to explicitly catch improperly formatted dates and log an error to the user rather than silently defaulting to zero.
- **Execution Time**: I optimized string stream extraction in handleEmail by swapping std::stringstream with a lightweight manual string search to extract the comma-delimited fields, avoiding stream initialization overhead.
- **Space Complexity**: I modified the local storage of intermediate strings during file parsing to reuse standard std::string buffers via std::move(), preventing the memory allocator from continually requesting new space for every loop iteration.
- **Maintainability**: I updated the prologue block to strictly match every required element in the assignment rubric, explicitly adding the "Revisions" line and ensuring the collaboration disclaimer met the exact academic guidelines. I also standardized variable names across all functions to adhere to a uniform camelCase naming convention.